

Problem Statement : Dropshipping Inventory

& Order Orchestrator

1. System Overview

The Dropshipping Inventory & Order Orchestrator is an e-commerce fulfilment platform that receives customer orders from an online store, verifies product availability with suppliers, coordinates multi-vendor fulfilment, creates and dispatches supplier purchase orders, tracks shipments, and handles backorder or exception conditions. The system provides order visibility for the store owner and reduces manual processing.

2. Components Identified

Component	Layer	Responsibility
Customer Web Interface	Presentation	Allows customers to place orders and view order/shipment status.
Admin Dashboard	Presentation	Provides store-owner visibility into orders, suppliers, shipments, and exceptions.
API Gateway / Backend Facade	Business	Single entry point for client requests and routing to internal services.
Order Management Service	Business	Receives/validates orders, manages order state, and coordinates fulfilment.
Supplier Management Service	Business	Maintains supplier information and supports supplier selection and purchase-order operations.
Inventory Service	Business	Queries supplier stock and compares availability with requested quantities.
Shipment Tracking Service	Business	Stores tracking information and updates shipment status.
Notification Service	Business	Sends customer/admin alerts for order, shipment, and exception events.
Database	Data	Stores orders, products, suppliers, inventory, shipments, users, and status information.
External Supplier Systems	Integration	Represents supplier APIs/systems for inventory and purchase-order communication.

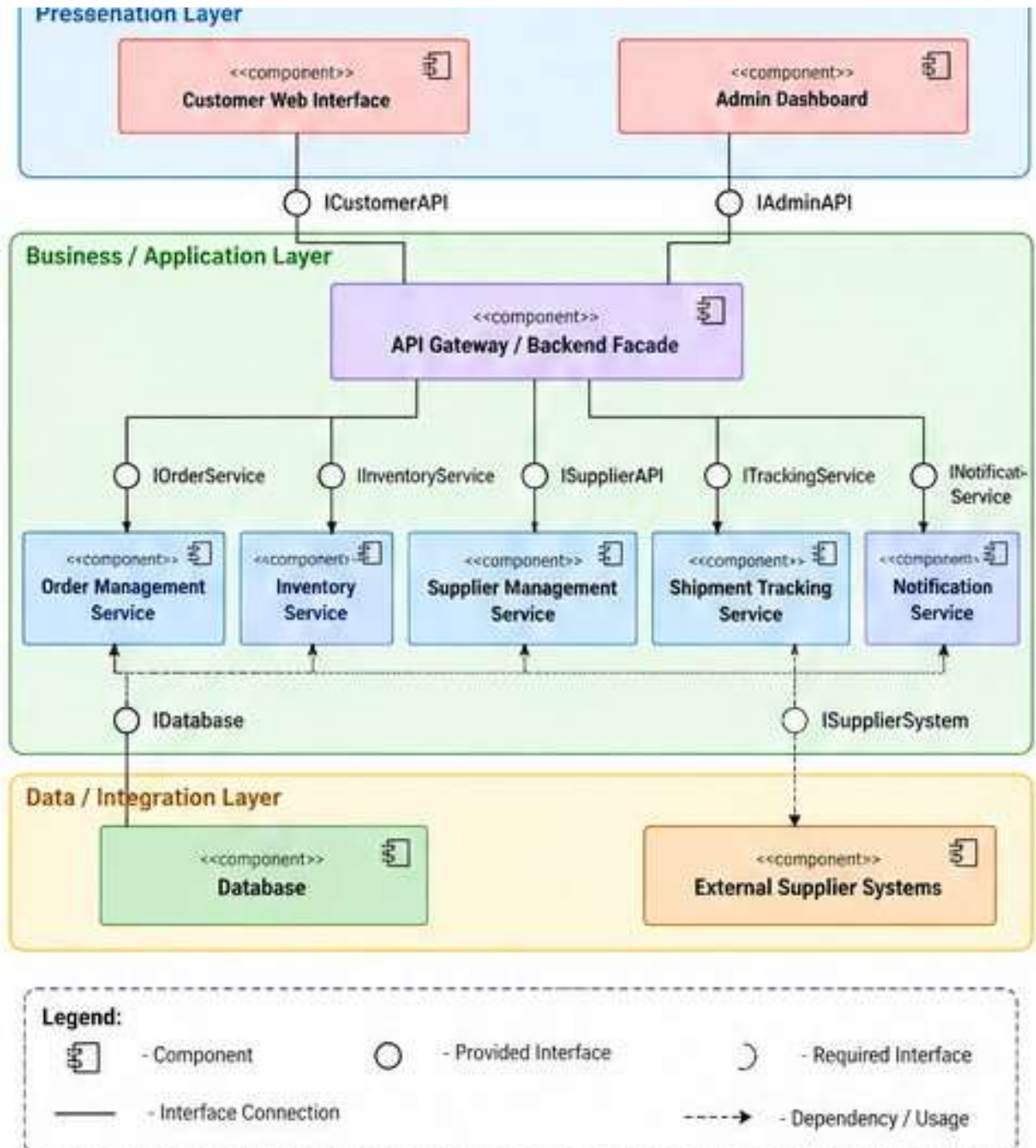
3. Interfaces

The following interfaces identify the principal services provided and required by the components in the UML model.

Interface	Provided by	Required by	Purpose
ICustomerAPI	API Gateway	Customer Web Interface	Handles customer requests such as browsing, order placement, and tracking.
IAdminAPI	API Gateway	Admin Dashboard	Handles admin requests for order, inventory, supplier, and shipment management.
IOrderService	Order Management Service	API Gateway	Processes customer orders and coordinates the fulfilment workflow.
ISupplierAPI	Supplier Management Service	Order Management Service	Provides supplier information and purchase-order creation services.
IInventoryService	Inventory Service	Order Management Service	Verifies supplier availability and compares stock with order quantity.
ITrackingService	Shipment Tracking Service	Order Management Service; Customer Web Interface	Provides tracking numbers, shipment status, and delivery updates.
INotificationService	Notification Service	Order Management Service	Sends alerts for order, shipment, and exception events.
IDatabase	Database	Order Management; Supplier; Inventory; Shipment; Notification	Provides persistent storage and retrieval of system data.
ISupplierSystem	External Supplier Systems	Supplier Management; Inventory Service	Connects the orchestrator to supplier APIs for inventory and fulfilment operations.

4. Component Diagram

Dropshipping Inventory & Order Orchestrator - UML Component Diagram



5. Architectural Justification

Architecture Selection: Microservices Architecture

Reason 1 - Independent service responsibilities

The system has distinct business functions - order management, supplier management, inventory verification, shipment tracking, and notifications. Separating these responsibilities keeps each module focused and easier to develop, test, deploy, and maintain.

Reason 2 - Scalability and fault isolation

Order ingestion, inventory queries, and shipment tracking can have different workloads. Independent services can be scaled separately, while failures in non-critical services such as notifications can be isolated from the core order workflow.

Security Advantage

Each service can enforce authentication and authorization at its API boundary. HTTPS, authenticated requests, role-based access, and protected database access reduce unauthorized order, supplier, or customer-data operations.

Performance Benefit

High-load services can be scaled independently. Inventory and tracking queries can be optimized or cached, while notifications can be processed asynchronously so non-critical work does not block the main order workflow.

Final Decision

Microservices aligns with multi-supplier integrations, order orchestration, real-time tracking, and exception handling. Layered architecture is simpler for small systems, while client-server centralizes control; microservices better supports independently evolving capabilities in this model.

Architecture comparison summary

Criteria	Layered Architecture	Client-Server	Microservices
Structure	Presentation / Business / Data layers	Central server + clients	Independent services
Scaling	Mostly whole-system scaling	Centralized bottleneck risk	Independent service scaling
Fault isolation	Moderate	Lower	Higher
Deployment	Often together	Centralized	Independent
Complexity	Lower	Lower	Higher operationally